MMOT225: Bulut Bilişim
Hafta 4: Tekrar - Lab - Linux VM Üzerine Docker Kurulumu ve Yönetimi

Dr. Öğr. Üyesi Hasan Oğuz

İstanbul Okan Üniversitesi | Meslek Yüksekokulu

Genel Tekrar: Bulut Bilişim Temel Yaklaşımlar

1) Geleneksel (On-Premise)

- Sunucular kurum içinde.
- Donanım satın alma + bakım sorumluluğu.
- Yüksek maliyet, düşük esneklik.

2) Sanallaştırma

- 1 fiziksel makine → çoklu VM.
- Kaynak paylaşımı (CPU, RAM, Disk).
- Daha verimli donanım kullanımı.

3) Bulut Bilişim

- Kaynaklar internet üzerinden.
- İsteğe bağlı başlatma (dakikalar içinde).
- "Kullandığın kadar öde" modeli.

Günlük Hayat

- Drive, Netflix, e-Devlet vb.
- Veri merkezi altyapısı arka planda.

Temel Mantık

Sanallaştırma = **Donanımı verimli bölmek**
Bulut = **Bu kaynakları internetten hizmet olarak almak**

Genel Tekrar: Sanallaştırma ve Temel Altyapı

1. Sanallaştırma ve Hipervizör

- **Sanal Makine (VM):** Donanımı paylaşan, kendi OS'ine sahip izole çalışma ortamı.
- **Hipervizör:** Kaynakları (CPU, RAM, Disk) paylaştıran kontrol yazılımı.
  - **Type 1 (Bare Metal):** Doğrudan donanıma kurulur (Sunucu tipi, yüksek performans).
  - **Type 2:** İşletim sistemi üzerine kurulur (Test/Eğitim tipi, örn: VirtualBox).
- **İzolasyon:** Bir VM'in çökmesi veya hacklenmesi diğerlerini ve Host'u etkilemez.

2. Kazanımlar ve Lab Özeti

- **Avantajlar:** Daha az fiziksel sunucu, düşük enerji maliyeti, hızlı ölçekleme ve donanım bağımsızlığı.
- **Sınırlamalar:** Host donanımı arızalanırsa tüm VM'ler kapanır. Aşırı kaynak tahsisi (overcommit) performansı düşürür.
- **Snapshot & Altyapı:** Riskli işlemler öncesi sistemin dondurulması hayat kurtarır. Bu izole altyapı, **Docker ve Kubernetes** gibi bulut sistemlerinin temelini oluşturur.

Genel Tekrar: Altyapı ve Sanal Diskler

Sanal Makine Anatomisi ve Snapshot

- **VM = Dosyalar Bütünü:** Sanal makine fiziksel bir donanım değil, konfigürasyon (`.vmx`/`.vbox`) ve sanal disk (`.vmdk`/`.vdi`) dosyalarından ibarettir.
- **Snapshot:** Sistemin belirli bir andaki durumunu dondurur. Bir **yedekleme (backup) yöntemi değildir**, riskli işlemler öncesi hızlı geri dönüş noktasıdır.

Thick Provisioning (Kalın Tahsis)

- Disk kapasitesi **peşin** ayrılır.
- Yüksek I/O performansı sağlar.
- Kritik veritabanları için idealdir.

Thin Provisioning (İnce Tahsis)

- Kapasite **kullandıkça** (dinamik) büyür.
- Over-provisioning (aşırı tahsis) imkanı verir.
- Geliştirme/test ortamları için idealdir.

Genel Tekrar: Sanal Ağ Modelleri Karşılaştırması

Sanal makinelerin fiziksel dünyayla nasıl haberleştiğini belirleyen 3 temel model:

Ağ Modeli	IP Alma Mantığı	Kullanım Amacı ve Özellikleri
1. NAT	Hipervizör iç ağ yaratır, dışarı Host'un IP'si ile çıkar.	**Varsayılan moddur.** VM internete girer ama dışarıdan VM'e doğrudan ulaşılamaz (Güvenli).
2. Bridged	Doğrudan fiziksel ortamdaki modemden/routerdan IP alır.	VM fiziksel ağın tam üyesi olur. Dışarıdan doğrudan erişilebilir (Sunucu olarak hizmet vermek için).
3. Host-Only	İnternet kapalıdır. Sadece hipervizör içinde IP dağıtılır.	**Tam izolasyon.** Zararlı yazılım analizi veya kapalı devre veritabanı testleri için kullanılır.

Genel Tekrar: Bulut Karakteristiği ve Konteynerler

Bulutun 5 Temel Özelliği (NIST)

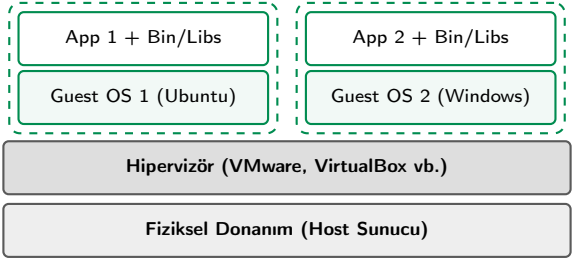
1. **On-Demand:** Anında self-servis kaynak.
2. **Geniş Ağ Erişimi:** Her cihazdan ulaşılabilirlik.
3. **Kaynak Havuzu:** Fiziksel kaynakların ortak kullanımı (Multi-tenant).
4. **Hızlı Esneklik:** İhtiyaca göre otomatik büyüme/küçülme.
5. **Ölçülebilir Hizmet:** Kullandığın kadar öde mantığı (Metrikler).

Neden Konteyner (Docker)?

- **Ortak Kernel:** Sanal makineler (VM) gibi her seferinde ağır bir misafir işletim sistemi (Guest OS) çalıştırmaz; alttaki Host OS'in çekirdeğini paylaşır.
- **Hız ve Boyut:** Gigabaytlar yerine Megabaytlar boyutundadır. Dakikalar değil saniyeler/milisaniyeler içinde başlar.
- **Taşınabilirlik:** "Benim bilgisayarımda çalışıyordu" problemini çözer.

Sanal Makine (VM) Nedir?

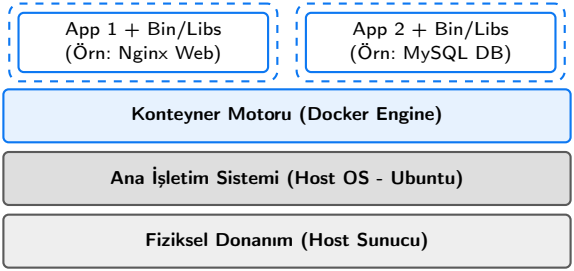
Sanal Makine (Virtual Machine), fiziksel donanım kaynaklarının bir **Hipervizör** aracılığıyla bölünerek, üzerinde tam bağımsız işletim sistemlerinin (Guest OS) çalıştırıldığı yapıdır.



- Her VM'in kendisine ait bir çekirdeği (kernel) ve dosya sistemi vardır.
- Tam izolasyon sağlar ancak **kaynak tüketimi çok yüksektir** (Örn: Sadece boş bir işletim sistemi bile 1-2 GB RAM harcar).

Konteyner (Container) Nedir?

Konteyner, donanımı değil, **işletim sistemini sanallaştıran** yapıdır. Tüm konteynerler alt katmandaki ana işletim sisteminin (Host OS) çekirdeğini doğrudan paylaşır.



- İçerisinde ağır bir "Guest OS" barındırmaz, bu yüzden boyutları Megabayt (MB) seviyesindedir.
- **Saniyeler içinde başlar** ve sunucuya neredeyse sıfır ek yük (overhead) bindirir.

Neden Konteyner? (VM vs. Konteyner Karşılaştırması)

Sistemleri geliştirirken ve buluta taşırken neden geleneksel Sanal Makineler yerine Konteyner (Docker) mimarisine geçiş yapıyoruz?

Özellik	Sanal Makine (VM)	Konteyner (Docker)
İşletim Sistemi	Her VM'de ayrı bir Guest OS bulunur	Alt sistemin (Host OS) Kernel'i paylaşılır
Başlatma Süresi	Dakikalar sürer	Saniyeler/Milisaniyeler sürer
Boyut	Gigabaytlarca (GB) alan kaplar	Megabaytlarca (MB) alan kaplar
Taşınabilirlik	Donanım bağımlılıkları sorun yaratabilir	"Benim bilgisayarımda çalışıyordu" sorunu biter

Özet Çıkarım

Laboratuvarda bir fiziksel bilgisayarda kaynak yetersizliğinden en fazla 4-5 VM açabilirken, aynı donanımda **onlarca** Konteyneri aynı anda sorunsuz çalıştırabiliriz.

Popüler Konteyner İmajları 1/2: Altyapı ve Veritabanı

Geliştiriciler, altyapı bileşenleri için tekerleği yeniden icat etmek yerine Docker Hub'daki **resmi (official)** ve optimize edilmiş imajları kullanırlar.

1. Web Sunucuları ve Ters Vekil (Reverse Proxy)

- **Nginx:** Sadece bir web sunucusu değil; yük dengeleyici (load balancer) ve ters vekildir. Mikroservislerin dış dünyaya açılan kapısıdır.
- **Httpd (Apache):** Geleneksel web barındırma süreçlerinin vazgeçilmezi.
- **Traefik:** Docker ve Kubernetes ortamları için tasarlanmış, dinamik yönlendirme yapan modern ters vekil.

2. Veritabanları ve Önbellek (Cache) Sistemleri

- **PostgreSQL & MySQL:** RDBMS standartları. Veriler konteynerin içinde değil, dışarıdan bağlanan *Volume* üzerinde tutulur.
- **Redis:** Bellek içi (in-memory) veri deposu. Oturum yönetimi ve önbellekleme için saniyede milyonlarca işlem yapar.
- **MongoDB:** NoSQL pazar lideri. Esnek belge tabanlı yapı ve büyük veri (Big Data) loglamaları için idealdir.

Popüler Konteyner İmajları 2/2: İşletim Sistemleri ve Mikroservisler

3. Minimal İşletim Sistemleri (Base Images)

- **Alpine Linux:** Konteyner dünyasının gizli kahramanı (∼5 MB). Güvenliği yüksek, ağı/diski korur. Çoğu modern imaj bunu taban alır.
- **Ubuntu / Debian:** Geleneksel Linux komut seti ve geniş kütüphane desteği arayanlar için temel imajlar.

4. Dil ve Çalışma Ortamları (Runtimes)

- **Node.js:** JS tabanlı backend ve frontend derlemeleri.
- **Python:** Veri bilimi, AI araçları ve web backend.
- **Golang (Go):** Derlenmiş çalıştırılabilir dosya (binary) ürettiği için "Scratch" (boş) konteynerlerde yüksek performans gösterir.

Laboratuvar İpucu: Mikroservis Mantığı

Modern bir e-ticaret sitesi tek bir makine değildir. Çeşitli görevler farklı konteynerlere dağıtılır:

- **Arayüz:** Önde trafiği karşılayan bir **Nginx** konteyneri.
- **Hız/Geçici Veri:** Sepet işlemlerini bellekte anında işleyen bir **Redis** konteyneri.
- **Kalıcı Veri:** Kullanıcı ve sipariş bilgilerini diske yazan bir **PostgreSQL** konteyneri.

Bu üç konteyner aynı sanal ağda birbirleriyle konuşarak çalışır.

Uygulama Öncesi Hazırlık: Sanal Makineyi Başlatma

Bugün, geçen hafta VirtualBox/VMware üzerinde kurduğumuz **Ubuntu Linux** sanal makinemizi bir Konteyner Sunucusuna (Docker Host) dönüştüreceğiz.

Adım 1: VM'i Başlatın

- Geçen hafta kurduğunuz Ubuntu VM'i çalıştırın ve giriş yapın.
- Sanal makinenin internete çıkabildiğinden emin olun (Tarayıcıdan bir siteye girerek test edebilirsiniz).

Adım 2: Terminali Açın

- Ubuntu masaüstünde **Ctrl** + **Alt** + **T** kısayolunu kullanarak veya menüden aratarak **Terminal**'i açın.
- Tüm işlemlerimizi bu siyah ekranda, komut satırı üzerinden gerçekleştireceğiz.

Adım 1: Ubuntu Üzerine Docker Motoru Kurulumu

Linux depolarından resmi Docker paketini indireceğiz. Terminale sırasıyla aşağıdaki komutları yazın ve şifreniz sorulduğunda girin (Şifre yazarken ekranda karakter görünmez, normaldir).

Kurulum Komutları

1. Paket listelerini güncelleyin:

```
$ sudo apt update
```

2. Docker'ı kurun:

```
$ sudo apt install docker.io -y
```

Servisi Başlatma ve Kontrol

Docker kurulduktan sonra arka planda çalıştığından emin olalım:

`$ sudo systemctl enable --now docker`

`$ sudo docker --version` *(Docker versiyonunu ekranda görmelisiniz)*

Adım 2: İlk Konteynerimizi Çalıştıralım (Hello World)

Docker Engine hazır. Şimdi sistemin dış dünyayla konuşabildiğini test edelim.

Komut: Docker'ı Test Etme

```
$ sudo docker run hello-world
```

Arka planda ne oldu?

1. Docker, sanal makinenizde `hello-world` imajını aradı. Bulamadı.
2. İnternete çıkıp bu imajı **Docker Hub**'dan indirdi (Pulling from library...).
3. İmajdan yeni bir **Konteyner** oluşturdu ve çalıştırdı.
4. Konteyner ekrana bir mesaj yazdırdı ve görevini tamamlayıp kendini kapattı.

Adım 3: Gerçek Bir Uygulama (Nginx Web Sunucusu)

Şimdi sadece tek satır kod ile saniyeler içinde çalışan bir Web Sunucusu ayağa kaldıracağız.

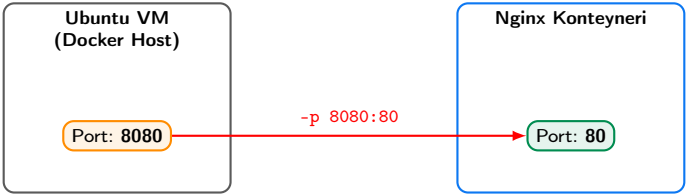
Komut: Arka Planda Nginx Çalıştırma

```
$ sudo docker run -d -p 8080:80 --name benim-sitem nginx
```

Parametrelerin Anlamları:

- `-d` (Detach): Konteyneri arka planda çalıştır. Terminali bize geri ver.
- `--name benim-sitem`: Konteynere rastgele bir isim vermek yerine biz isim atıyoruz.
- `nginx`: Kullanılacak imajın adı (Docker Hub'dan resmi Nginx imajı).
- `-p 8080:80` **(Port Yönlendirme):** En önemli kısım!

Kritik Kavram: Port Yönlendirme (Port Mapping)



Konteynerler kendi içlerinde izoledir. Nginx varsayılan olarak 80 portunda çalışır. `-p 8080:80` diyerek **"Ubuntu VM'in 8080 portuna gelen istekleri, konteynerin 80 portuna ilet"** diyoruz.

Test Edin

Ubuntu içindeki Firefox tarayıcısını açın ve adres çubuğuna şunu yazın: `http://localhost:8080`

Adım 4: Konteynerleri Yönetmek (Durdurmak ve Silmek)

Uygulamamız çalışıyor. Peki bunu nasıl kapatacağız? Terminale geri dönün:

1. Çalışanları Listeleme

```
$ sudo docker ps
```

Aktif olan konteynerleri, ID'lerini ve port durumlarını listeler.

2. Konteyneri Durdurma

```
$ sudo docker stop benim-sitem
```

Konteyner durur. Tarayıcıda sayfayı yenilediğinizde sitenin kapandığını görün.

3. Tüm Konteynerleri Görme

```
$ sudo docker ps -a
```

Durmuş olanlar (örneğin hello-world) dahil tüm konteynerleri listeler.

4. Konteyneri Tamamen Silme

```
$ sudo docker rm benim-sitem
```

Konteyneri sistemden tamamen temizler. Sadece çalışan kopyası silinir, imaj kalır.

Kontrol Listesi ve Teslim (Çıktılar)

Öğrenci Kontrol Listesi

- Ubuntu terminali üzerinden `docker.io` kuruldu.
- `sudo docker run hello-world` başarıyla çalıştırıldı.
- `nginx` imajı ile arka planda (`-d`) bir web sunucusu ayağa kaldırıldı.
- Ubuntu içindeki Firefox'tan `localhost:8080` portuna girilerek Nginx karşılama ekranı görüldü.
- Konteyner komut satırından durduruldu (`stop`) ve silindi (`rm`).

Teslim Edilecekler (Ödev)

- **3 Ekran Görüntüsü:**
  1. `docker run hello-world` terminal çıktısı.
  2. Firefox'ta görünen **Nginx** ekranı (Adres çubuğu dahil).
  3. `docker ps` komutunun çıktısı (Nginx çalışırken).
- **Kısa Cevap:** "-p 8080:80" komutu tam olarak ne işe yarar?

Laboratuvar Sonu - Sorularınız?